

WHATSAPP-LIKE MESSAGING SYSTEM

Software Requirements Specification (SRS)

Trainee Implementation Guide

Version 1.0 | C++ Object-Oriented Programming Training Module

Prepared for: Software Engineering Trainees

Language: C++ (Classes, Inheritance, Polymorphism, STL Containers)

INTERNAL TRAINING DOCUMENT — NOT FOR PUBLIC DISTRIBUTION

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) document defines the complete functional and design requirements for implementing a WhatsApp-like messaging system in C++. It is intended to guide trainees through completing a partially-written codebase by specifying exactly what each class and method must do.

By the end of this exercise, trainees will demonstrate mastery of:

- Designing related classes with inheritance (a Chat base class with PrivateChat and GroupChat derived classes)
- Managing collections of objects with STL containers (vector<User>, vector<Message>, vector<Chat*>)
- Implementing constructors, getters, setters, and input validation logic
- Using pointers and virtual functions (polymorphic displayChat())
- Working with time functions from <ctime> to manage timestamps
- Writing clean, correct object-oriented code without state, memory, or index errors

1.2 Project Overview

The system is a standalone, console-based C++ application that simulates a messaging platform. It is composed of six related classes: User, Message, Chat (base), PrivateChat, GroupChat, and WhatsApp. The skeleton code — full class declarations with an empty body for each required method — is provided. Trainees must implement all method bodies as specified in Section 3. All data exists only in memory during runtime and is lost when the application closes.

1.3 Scope

The implementation covers the following six classes:

- User — stores account information (username, password, phone, status) and last-seen tracking
- Message — stores a single chat message, its sender, timestamp, status, and optional reply reference
- Chat (base) — stores participants and messages shared by both chat types; provides search and delete
- PrivateChat — a one-on-one conversation between exactly two users, with a typing indicator
- GroupChat — a multi-participant conversation with admins, a description, and join requests
- WhatsApp — the top-level application: manages all users, all chats, and the logged-in session

1.4 Definitions and Acronyms

Term	Definition
SRS	Software Requirements Specification.
User	A person using the application; represented by the User class.
Chat	A conversation between two or more users; represented by the Chat base class and its subclasses.
Message	A single unit of communication sent in a chat, represented by the Message class.
Timestamp	Date and time when an action occurred, obtained via <ctime>.
Admin	A participant in a GroupChat with elevated permissions (add admins, remove participants).
Trainee	A junior developer completing this exercise.

2. System Design Overview

2.1 Class Architecture

The WhatsApp class sits at the top of the object graph: it owns a `vector<User>` of all registered accounts and a `vector<Chat*>` of all active conversations. Chat is an abstract base storing shared state (participants and messages); `PrivateChat` and `GroupChat` inherit from it and override `displayChat()` to render themselves differently. Message objects live inside each Chat's message vector, and a Message may hold a pointer back to another Message it is replying to.

2.2 Data Storage

All data is held in memory only — there is no file or database persistence. Users are stored by value in a `vector<User>`; chats are stored as pointers (`vector<Chat*>`) so that `PrivateChat` and `GroupChat` objects can be referenced polymorphically through their common Chat base type. All data is lost when the application closes.

2.3 Design Conventions

The implementation must follow these constraints:

- Target platforms: Windows, Linux, and macOS
- C++ compiler supporting C++11 or later
- Console/terminal interface only — no GUI or network I/O
- All classes must be implemented exactly as specified in Section 3
- No external libraries beyond the C++ standard library

3. Class Specifications

3.1 User Class

Data Members:

- `string username` — unique identifier for the user
- `string password` — user's password (stored as string for simplicity)
- `string phoneNumber` — contact number
- `string status` — Online/Offline/Custom status
- `string lastSeen` — timestamp of last activity

Constructor Requirements:

1. Default constructor: initialize with empty strings and current timestamp
2. Parameterized constructor: set username, password, phone, and update lastSeen

Method Requirements:

Method	Input	Output	Behavior
<code>getUsername</code>	None	string	Return username
<code>getPhoneNumber</code>	None	string	Return phone number
<code>getStatus</code>	None	string	Return current status

Method	Input	Output	Behavior
getLastSeen	None	string	Return last seen timestamp
setStatus	string newStatus	void	Update status
setPhoneNumber	string phone	void	Update phone number
updateLastSeen	None	void	Set lastSeen to current time
checkPassword	string pwd	bool	Compare with stored password
changePassword	string newPwd	void	Update password

3.2 Message Class

Data Members:

- string sender — username of sender
- string content — message text
- string timestamp — when message was sent
- string status — sent / delivered / read
- Message* replyTo — pointer to the message being replied to

Constructor Requirements:

1. Default: initialize with empty values and a null reply pointer
2. Parameterized: set sender, content, current timestamp, status="sent", replyTo=nullptr

Method Requirements:

Method	Input	Output	Behavior
getContent	None	string	Return message content
getSender	None	string	Return sender username
getTimestamp	None	string	Return timestamp
getStatus	None	string	Return status
getReplyTo	None	Message*	Return pointer to replied message
setStatus	string newStatus	void	Update status
setReplyTo	Message* msg	void	Set reply reference
updateTimestamp	None	void	Set to current time
display	None	void	Print formatted message
addEmoji	string emojiCode	void	Replace code with emoji symbol

3.3 Chat Class (Base)

Data Members:

- vector<string> participants — list of usernames in the chat

- vector<Message> messages — all messages in the chat
- string chatName — name of the chat

Constructor Requirements:

1. Default: empty vectors, empty name
2. Parameterized: set participants and name

Method Requirements:

Method	Input	Output	Behavior
addMessage	const Message& msg	void	Add message to vector
deleteMessage	int index, const string& username	bool	Delete if user is sender, return success
displayChat	None	void (virtual)	Display all messages
searchMessages	string keyword	vector<Message>	Return messages containing keyword

3.4 PrivateChat Class

Data Members:

- string user1 — first participant
- string user2 — second participant

Constructor Requirements:

1. Initialize with two users, set participants vector, chatName = "Chat between user1 and user2"

Method Requirements:

Method	Input	Output	Behavior
displayChat	None	void (override)	Show messages with typing indicator
showTypingIndicator	const string& username	void	Print "[username] is typing..."

3.5 GroupChat Class

Data Members:

- vector<string> admins — admin usernames
- string description — group description

Constructor Requirements:

1. Set participants and name, add the creator to both participants and admins

Method Requirements:

Method	Input	Output	Behavior
addAdmin	string newAdmin	void	Add to admins vector
removeParticipant	string admin, string userToRemove	bool	Admin removes user, return success

Method	Input	Output	Behavior
isAdmin	string username	bool	Check if user is admin
isParticipant	string username	bool	Check if user is in participants
setDescription	string desc	void	Update description
displayChat	None	void (override)	Show group info and messages
sendJoinRequest	const string& username	void	Request to join group

3.6 WhatsApp Class

Data Members:

- vector<User> users — all registered users
- vector<Chat*> chats — all active chats
- int currentUserIndex — index of the logged-in user (-1 if none)

Constructor Requirements:

1. No parameterized constructor required — members are default-initialized (empty vectors, currentUserIndex = -1)

Method Requirements:

Method	Input	Output	Behavior
findUserIndex	string username	int	Linear search, return index or -1
isLoggedIn	None	bool	Check currentUserIndex != -1
getCurrentUsername	None	string	Return username of current user
signUp	None	void	Register new user, check uniqueness
login	None	void	Authenticate user
startPrivateChat	None	void	Create chat with another user
createGroup	None	void	Create group with participants
viewChats	None	void	Display all of the user's chats
logout	None	void	Set currentUserIndex = -1

4. Edge Cases

Your implementation must correctly handle all of the following situations:

Situation	Expected Behavior	Why It Matters
Registering a username that already exists	Reject sign-up, show an error	Username must remain unique across the system (FR2)

Situation	Expected Behavior	Why It Matters
Password shorter than 6 characters	Reject sign-up, show an error	Enforces the minimum password length rule (FR3)
User tries to delete a message they did not send	deleteMessage returns false, no change made	Users may only delete their own messages (FR9, 5.3)
Creating a group with fewer than 2 participants	Reject group creation	Groups require a name and at least 2 participants (FR17)
Non-admin attempts to add an admin or remove a participant	Operation rejected / returns false	Only admins may manage group membership (FR19, FR20)
searchMessages finds no matches	Return an empty vector<Message>	Callers must handle empty results without crashing
User attempts to open a chat they are not part of	Access denied	Users cannot access chats they are not part of (5.3)
findUserIndex called with an unknown username	Return -1	Callers must check for -1 before indexing into users
Replying to a message, then the original message is deleted	replyTo pointer must not be dereferenced after deletion	Avoids dangling-pointer/undefined-behavior bugs

5. Method Summary

Class	Method	Signature	Purpose
User	getUsername / getPhoneNumber / getStatus / getLastSeen	string f() const	Read accessors
User	setStatus / setPhoneNumber	void f(string)	Update mutable fields
User	updateLastSeen	void f()	Refresh last-seen timestamp
User	checkPassword / changePassword	bool / void f(string)	Verify or update password
Message	getContent / getSender / getTimestamp / getStatus	string f() const	Read accessors
Message	getReplyTo / setReplyTo	Message* f() / void f(Message*)	Manage reply reference
Message	display / addEmoji	void f() / void f(string)	Render message / substitute emoji codes
Chat (base)	addMessage / deleteMessage	void / bool	Add or remove a message
Chat (base)	displayChat (virtual)	void	Render chat contents
Chat (base)	searchMessages	vector<Message> f(string)	Keyword search
PrivateChat	displayChat (override) / showTypingIndicator	void	Render 1:1 chat and typing status

Class	Method	Signature	Purpose
GroupChat	addAdmin / removeParticipant / isAdmin / isParticipant	void / bool	Manage membership
GroupChat	setDescription / displayChat (override) / sendJoinRequest	void	Group metadata and display
WhatsApp	findUserIndex / isLoggedIn / getCurrentUsername	int / bool / string	Session lookups
WhatsApp	signUp / login / logout	void	Session management
WhatsApp	startPrivateChat / createGroup / viewChats	void	Chat creation and listing

6. Feature Requirements & Expected Behavior

6.1 User Management

Description: Users can create accounts, log in, and manage their profiles.

Functional Requirements:

- FR1: Users must register with username, password, and phone number
- FR2: Usernames must be unique in the system
- FR3: Passwords must be at least 6 characters long
- FR4: Users can update their status and phone number
- FR5: Last seen timestamp updates when user logs out or performs actions
- FR6: Users can change their password

6.2 Messaging

Description: Users can send and manage messages in chats.

Functional Requirements:

- FR7: Messages must contain sender, content, and timestamp
- FR8: Messages can be sent in private or group chats
- FR9: Users can delete their own messages
- FR10: Messages have status (sent, delivered, read)
- FR11: Users can reply to specific messages
- FR12: Users can search messages by keyword
- FR13: Users can add emojis (represented as text codes like "😊" or "😂") to messages

6.3 Private Chats

Description: One-on-one conversations between two users.

Functional Requirements:

- FR14: Private chat created between exactly two users
- FR15: Show typing indicator when a user is typing

- FR16: Display chat with messages in chronological order

6.4 Group Chats

Description: Conversations with multiple participants.

Functional Requirements:

- FR17: Groups require a name and at least 2 participants
- FR18: Creator becomes admin automatically
- FR19: Admins can add other admins
- FR20: Admins can remove participants
- FR21: Groups can have descriptions
- FR22: Users can request to join groups
- FR23: Display all participants and admins when viewing a group

7. External Interface & Non-Functional Requirements

7.1 User Interfaces

- Console-based text interface
- Menu-driven navigation with numbered options
- Clear prompts for user input
- Formatted display of messages and chats

7.2 Software Interfaces

- Standard input/output streams
- Time/date functions from <ctime>

7.3 Communication Interfaces

- No network communication required
- All data stored in memory only

7.4 Performance

- Application must respond to user input within 2 seconds
- Support at least 100 users and 1000 messages during a session
- Search functionality must complete within a reasonable time

7.5 Reliability

- System should not crash on invalid input
- Error messages must be user-friendly
- Memory management must prevent leaks

7.6 Security

- Passwords must not be displayed when typing

- Users can only delete their own messages
- Users cannot access chats they are not part of

7.7 Usability

- Menu options must be clear and consistent
- Helpful error messages for invalid inputs
- Display current user and time information

8. Expected Test Output

Compile and run with:

```
g++ -std=c++11 -Wall -Wextra whatsapp.cpp -o whatsapp && ./whatsapp
```

8.1 User Registration (TC1)

Input: username="alice", password="pass123", phone="555-0101". Expected: User created; alice can subsequently log in with those credentials.

8.2 Private Chat (TC2)

Setup: alice and bob are logged in. Expected: A private chat can be created between them, messages can be sent, and both users see the message history.

8.3 Group Admin (TC3)

Setup: alice creates a group with members bob and charlie. Expected: alice is admin automatically and can add further admins or remove members; bob and charlie cannot.

8.4 Message Search (TC4)

Setup: search the keyword "hello" within a chat. Expected: searchMessages returns every message in that chat whose content contains "hello".

8.5 Message Reply (TC5)

Setup: send a message, then send a second message replying to it. Expected: the reply's getReplyTo() returns a pointer to the original message, and display() shows the reference.

9. Common Mistakes & Tips

Mistake	Why It's Wrong	Correct Approach
Skipping the uniqueness check in signUp	Two users could share a username, breaking findUserIndex and login	Always call findUserIndex first and reject sign-up if it does not return -1
Comparing password fields directly instead of using checkPassword	Bypasses encapsulation and is easy to get wrong when the field is later hashed	Always call checkPassword(pwd) rather than reading the field
Forgetting to call updateLastSeen on logout or activity	lastSeen becomes stale and FR5 is not satisfied	Call updateLastSeen() at the end of every action that should refresh it

Mistake	Why It's Wrong	Correct Approach
Storing chats by value instead of by pointer in WhatsApp	Slices PrivateChat/GroupChat down to the base Chat type, losing overridden behavior	Store chats as vector<Chat*> and allocate with new
Leaking allocated Chat objects	Every new Chat* without a matching delete leaks memory (violates 5.2)	Delete all chat pointers in the WhatsApp destructor
Not checking deleteMessage's username before removing	Any user could delete another user's message	Compare the message's sender to the requesting username before erasing
Off-by-one or missing bounds check in deleteMessage(index, ...)	Out-of-range index causes undefined behavior	Validate $0 \leq \text{index} < \text{messages.size}()$ before accessing
Dereferencing replyTo after the original message is deleted	Leaves a dangling pointer	Clear or null out replyTo references when the target message is deleted

10. Implementation Checklist

Use this checklist to track your progress. Every item must be completed before submission.

10.1 Phase 1: User Class

#	Item	Done?
1	Constructors initialize all fields correctly, including lastSeen	☐
2	All getters and setters implemented	☐
3	updateLastSeen uses <ctime> functions	☐
4	Password validation (minimum 6 characters) enforced at sign-up	☐

10.2 Phase 2: Message Class

#	Item	Done?
5	Constructors implemented (default and parameterized)	☐
6	display() prints a correctly formatted message	☐
7	addEmoji replaces recognized codes with the mapped emoji	☐
8	Status management (sent/delivered/read) implemented	☐

10.3 Phase 3: Chat Classes

#	Item	Done?
9	Chat base class: addMessage, deleteMessage, searchMessages implemented	☐
10	deleteMessage checks sender before removing	☐
11	PrivateChat: displayChat override and typing indicator complete	☐
12	GroupChat: admin management (add/remove/isAdmin/isParticipant) complete	☐

#	Item	Done?
13	GroupChat: displayChat override shows participants, admins, and messages	☐

10.4 Phase 4: WhatsApp App

#	Item	Done?
14	signUp checks username uniqueness before registering	☐
15	login authenticates against stored credentials	☐
16	startPrivateChat and createGroup construct chats correctly	☐
17	viewChats lists all chats the current user participates in	☐
18	logout resets currentUserIndex to -1	☐

10.5 Phase 5: Testing and Refinement

#	Item	Done?
19	All edge cases in Section 4 verified	☐
20	Invalid input handled without crashing	☐
21	All Chat* allocations are released (no leaks)	☐
22	Sample test cases (Section 8) pass	☐

11. Grading Criteria

Criterion	Weight / Description
Correctness of Functionality	40% — All classes behave as specified; functional requirements FR1–FR23 satisfied
Edge Case Handling	25% — Duplicate usernames, invalid passwords, unauthorized deletes, group-size limits, and other cases in Section 4 handled correctly
Class Design & Structure	20% — Correct use of inheritance and polymorphism (Chat/PrivateChat/GroupChat), proper encapsulation, no memory leaks
Code Quality	10% — Readable code, consistent naming, no unnecessary copies or dead code
Compilation	5% — Compiles without warnings: g++ -std=c++11 -Wall -Wextra

Appendix — Quick Reference Card

A concise summary for quick lookup during implementation.

Class & Key Method Signatures

Class	Key Signature	Returns
User	bool checkPassword(string pwd)	true if pwd matches stored password
Message	void addEmoji(string emojiCode)	Modifies content in place
Chat (base)	bool deleteMessage(int index, const string& username)	true if deleted
Chat (base)	vector<Message> searchMessages(string keyword)	Matching messages
PrivateChat	void showTypingIndicator(const string& username)	void
GroupChat	bool removeParticipant(string admin, string userToRemove)	true if removed
WhatsApp	int findUserIndex(string username)	Index or -1

Key Rules at a Glance

- Username uniqueness: reject signUp if findUserIndex(username) != -1
- Password rule: length >= 6 characters
- Group creation: requires a name and at least 2 participants; creator becomes admin
- Message ownership: deleteMessage succeeds only if username == message.getSender()
- Session state: currentUserIndex == -1 means logged out
- Emoji map: ":" -> "😊", ":(" -> "😞", ":D" -> "😄", "<3" -> "❤️", ":thumbsup:" -> "👍"

End of Document